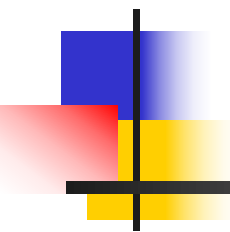


C语言与程序设计

The C Programming Language



第11章 复杂类型的指针

毛伏兵

华中科技大学计算机学院

11.1 指向数组的指针

11.1.1 指向数组的指针的声明与定义

指向数组的指针又称为数组的指针。

类型名 (*标识符) [常量];

↑ ↑ ↑

所指数组元素的类型 指针变量名 所指数组的长度

11.1.2 用数组名间访二维数组的元素

二维数组被看成以1维数组（行）为元素的一维数组；

```
int u[2][3]={1, 3, 5  
             {2, 4, 6}};
```

$u \rightarrow$

$u[0]$	1	3	5
$u[1]$	2	4	6

u 被看成有两个1维数组（行）元素($u[0], u[1]$)组成的一维数组

$u[0]$ 第0行首地址 即 $u[0] == \&u[0][0]$

$u[1]$ 第1行首地址 即 $u[1] == \&u[1][0]$

i 行 j 列的元素的地址？

(1) 用指向数组元素的指针表示: $u[i]+j$

(2) 用指向一维数组的指针表示: $*(u+i)+j$

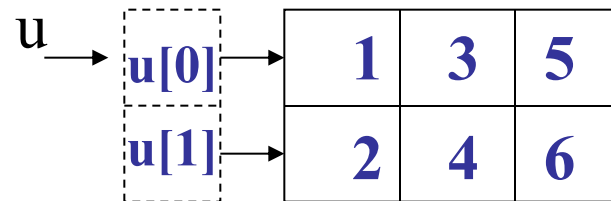
如何间访 i 行 j 列的元素？

(1) $*(u[i]+j)$

(2) $*(*(u+i)+j)$

二维数组元素的表示方法

```
int u[2][3]={ {1, 3, 5}  
              {2, 4, 6} };
```



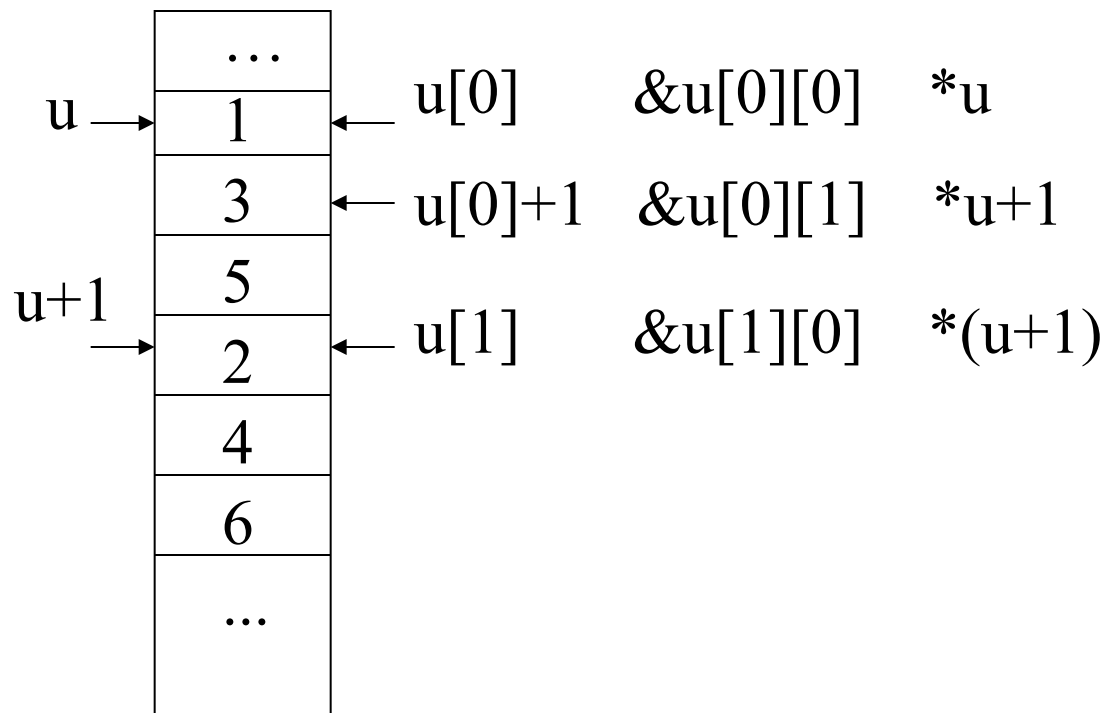
1、下标法 `u[i][j]`

2、指针法 `* ( u[i]+j )` `u[i]: int *`

`* ( *(u+i)+j )` `u: int (*)[3]`

用指针变量描述二维数组元素

```
int u[2][3]={  
    {1, 3, 5}  
    {2, 4, 6}  
};
```



1、将指针定义为指向数组元素

见8.4.3

2、将指针定义为指向由m个元素组成的数组的指针

```
int a[3][2]={1, 3}, {4, 6}, {7, 9};
```

```
int (*p)[2];
```

p是指向数组的指针，该数组有2个int型元素

```
p=a;
```

<code>*(p)</code> 或 <code>(p)[0]</code>	<code>*(p+1)</code> 或 <code>(p)[1]</code>
<code>*(p+1)</code> 或 <code>(p+1)[0]</code>	<code>*(p+1+1)</code> 或 <code>(p+1)[1]</code>
<code>*(p+2)</code> 或 <code>(p+2)[0]</code>	<code>*(p+2+1)</code> 或 <code>(p+2)[1]</code>

例 二维数组元素的输入/输出

```
#include <stdio.h>
```

```
#define I 2
```

```
#define J 3
```

```
void main(void)
```

```
{  int u[I][J], (*p)[J]=u;
```

```
    int j;
```

```
    for(j=0;j<J;j++) /* 用指向数组元素的指针完成第0行元素的输入 */
```

```
        scanf("%d", (u[0]+j));
```

```
    for(p++,j=0;j<J;j++) /* 用指向数组的指针完成第1行元素的输入 */
```

```
        scanf("%d", (*p+j));
```

```
    for(j=0;j<J;j++) /* 用指向数组的指针完成第0行元素的输出 */
```

```
        printf("%6d", *(u+j));
```

```
    printf("\n");
```

```
    for(j=0;j<J;j++) /* 用指向数组元素的指针完成第1行元素的输出 */
```

```
        printf("%6d", *(u[1]+j));
```

```
    printf("\n");
```

```
}
```

11.1.3 用指向数组的指针表示三维数组

类推：

三维数组被看成以二维数组（页）为元素的一维数组

一个n维数组被看成以n-1维数组为元素的一维数组。

```
int x[2][3][4];
```

```
int (*p2)[3][4]=x;
```

/*p2是指向**3行4列的二维整型数组**的指针*/

```
*(*(*(p2+i)+j)+k) <==> p2[i][j][k]
```


11.1.4 二维数组作函数参数

- 形参说明为数组

```
fun(int x[ ][4])
```

```
{
```

- 形参说明为指针

```
...
```

```
}
```

{ 指向数组元素的指针

{ 指向下一级数组的指针

```
fun(int *x)
```

```
{
```

```
...
```

```
}
```

```
fun(int (*x)[4] )
```

```
{
```

```
...
```

```
}
```

11.2 用typedef定义类型表达式

11.2.1 类型表达式

C中的表达式可以分成两类。

- (1) **值表达式**，由运算符和操作数组成，可被CPU处理和计算
- (2) **类型表达式**，由类型说明符和数据类型名组成，类型说明符有：()、[]、*

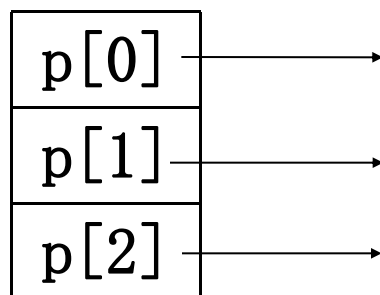
int (*) [5]

11.2.2 用typedef定义类型表达式

类型表达式可以出现在两个地方：

1) 声明语句中

```
int *p[3];
```



2) typedef定义中

typedef是关键字，为一个类型表达式定义一个别名。

typedef 类型区分符 说明符 ;



基本类型 结构 联合

也可以是由 **typedef** 定义的类型名

(1) **typedef unsigned int size_t;**

size_t定义为**unsigned int**类型

size_t x, y; /* unsigned int x, y; */

(2) **typedef char *string;**

/ 将 string 定义为 char 型指针 */*

string p, s[10];  **char *p, *s[10];**

/ p是字符指针,*

*s 是含有10个元素的字符指针数组 */*

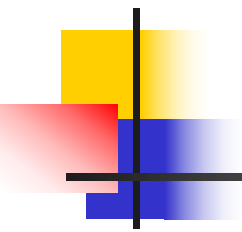
而 **#define string char ***

/ string 是宏名，简单的串替换 */*

string p, s[10];  **char *p, s[10];**

/ p是字符指针,*

*s 是含有10个元素的字符数组 */*



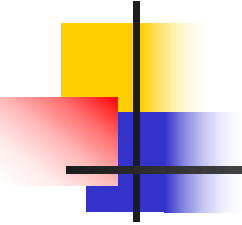
(3) typedef struct {

 char num[12]; /* 学号成员，字符数组类型 */

 char name[9]; /* 姓名成员，字符数组类型 */

} student;

student a,b;

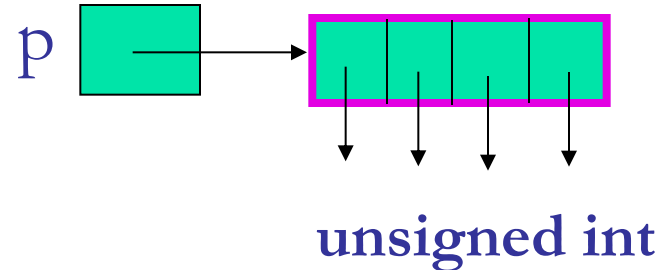


(4) `typedef char * (*p_to_fun)(char *,char *);`
p_to_fun 定义为 `char *(*)(char *,char *)`

`p_to_fun` fptr;

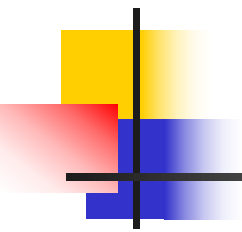
11.3 复杂说明的解释

*unsigned *(*p)[4]*



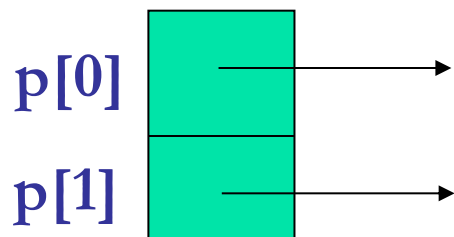
*p*是指向数组的指针，
该数组有4个无符号整型指针元素。
或

*p*是有4个无符号整型指针元素的数组的指针。



```
char *(*p[2])(char *, int);
```

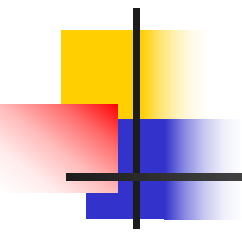
*p*是含有2个指针元素的数组，每个指针指向有一个字符指针参数和一个整型参数，返回值为字符型指针的函数。



函数：*char **(***char ****, ***int***)

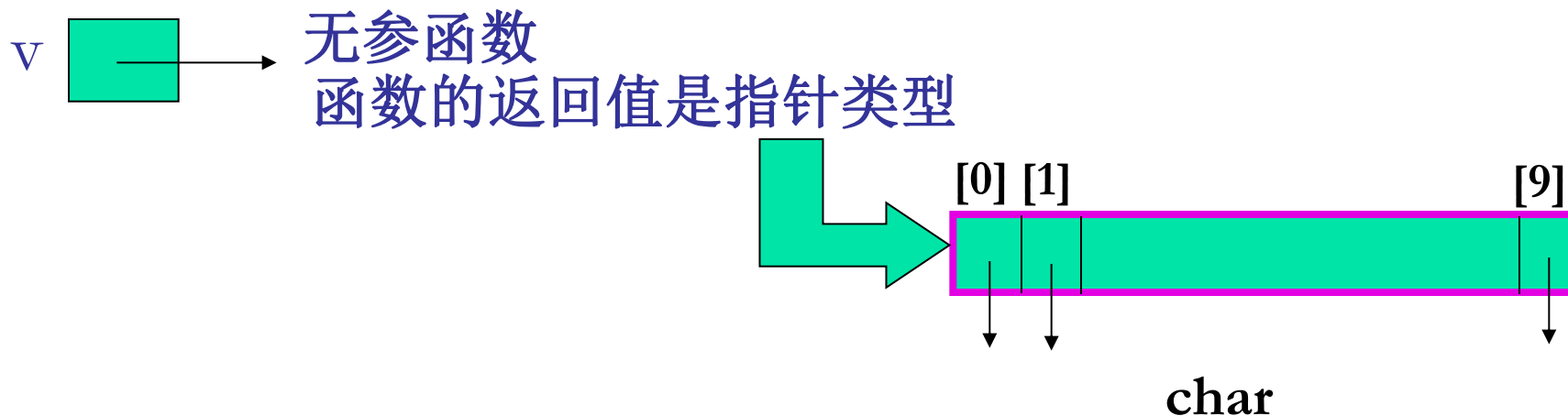
例11.7 有两个元素的函数指针数组的复杂说明的使用举例。

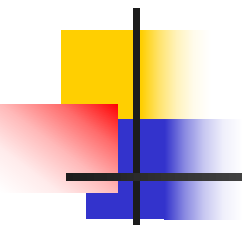
```
#include "stdio.h"
int a1(int);
int a2(int);
int main(void)          /*fpa是有2个元素的函数指针数组，每个元素指向*/
{  int i,(*fpa[2])(int); /*的函数有一个整型形参，返回整型值*/
    fpa[0]=a1;   /*第1个（下标为0）元素指向a1函数*/
    fpa[1]=a2;   /*第2个（下标为1）元素指向a2函数*/
    for(i=0;i<2;i++)
        printf("%d\n",fpa[i]((i+1)*5));/*依次调用fpa[0]、fpa[1]所指函数*/
    return 0;
}
int a1(int x){ printf("in function a1,x=%d\n",x); return 2*x; }
int a2(int y){ printf("in function a2,y=%d\n",y); return 2*y; }
```



```
char * (3) (4)
      (6) (5) (1) (2) (*(*v)(void))[10];
```

v 是函数的指针，该函数没有参数，
返回值是指向有10个元素的字符指针数组的指针。





```
int (*f(char *(*)(int)))[10];
```

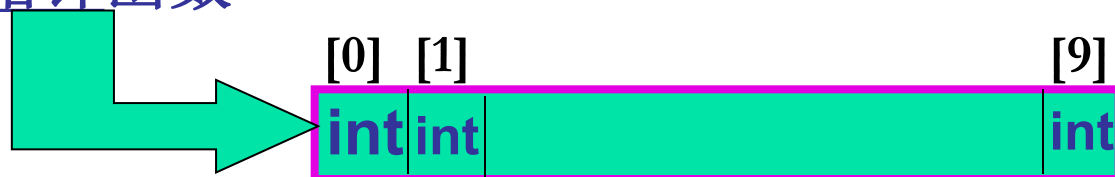
*f*是一个指针函数，

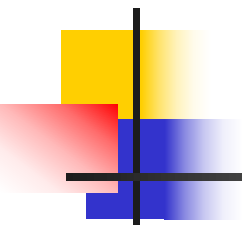
*f*函数的形参为一个指向函数的指针，所指函数有一个整型形参且返回值类型为char *；

*f*函数的返回值是指向有10个整型元素的数组的指针。

f 的形参： char *(*)(int)

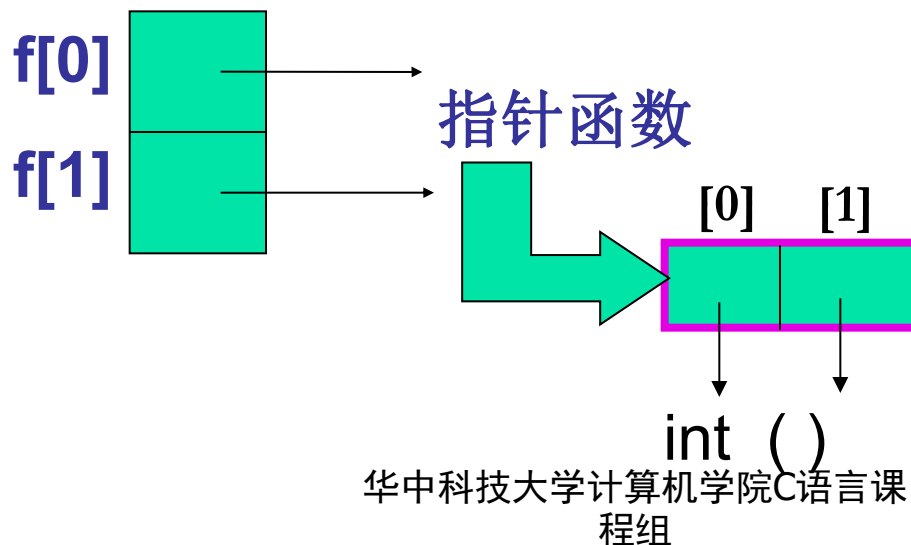
f是一个指针函数

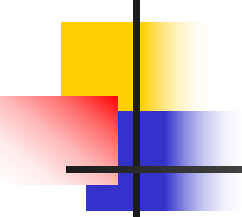




```
int ( *(*(*f[2])0)[2])0;
```

f是一个有两个元素的函数指针数组；所指函数的返回值是指向有两个元素的指针数组的指针，指针数组的每个指针元素指向一个无参整型函数。（无参整型函数指函数没有参数且返回值为整型值）。





****11.4 复杂说明的应用**

- **例11.6** 设p是一个函数指针，所指向的函数无参，且返回值为一个函数指针；该（返回的）函数指针指向的函数有两个字符指针形参，且返回值为字符指针值。请写出该声明语句并编写使用p的应用程序。
- 相应的声明语句为：

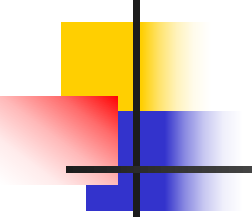
char *(*(*p)(void))(char *,char *);

```

#include "stdio.h"
char *(*p)(void)(char *,char *);/*f1是指针函数，返回的指针指向有两个*/
char *(*f1(void))(char *,char *);/*字符指针形参，返回值为字符指针值的函数*/
char *strcpy(char *,char *);/*有两个字符指针形参，返回值为字符指针值的函数*/
int main(void)
{
    char a[80],b[]="aaa,bbb,ccc",*str;
    p=f1;/*p指向f1，下面语句先计算(*p)()，即f1函数*/
    str=(*p)()(a,b);/*返回strcpy；然后调用strcpy(a,b)，结果赋给str*/
    printf("%s\n",str);
    return 0;
}
char *(*f1(void))(char *,char *)
{
    /*f1(void)表示f1是无参函数，其余部分都是描述它的返回值的类型*/
    return strcpy; /*返回指向有两个字符指针形参，返回值为字符指针值的函数*/
}
char *strcpy(char *t,char *s)
{
    char *p=t;
    while((*t++=*s++)!='\0');
    return p;
}

```

演示： [源程序\ex11_6.c](#)



复杂类型的使用

- 下面进一步就复杂说明：

`int (*fpa[2])(int);`

和`int ((*(*f[2])())[2])(int,int);`

为例，说明复杂类型的使用。

- 对于复杂说明`int (*fpa[2])(int);`要从理解fpa的含义，对fpa数组的元素进行赋值，以及通过fpa去调用fpa数组元素所指函数三个方面加以把握。
- 由于fpa是一个有两个元素的函数指针数组，因此构造两个函数a1和a2，并通过`fpa[0]=a1;`和`fpa[1]=a2;`使fpa[0]指向a1，fpa[1]指向a2。
- 同时，通过表达式`fpa[i]((i+1)*5)`，当i=0调用函数a1，当i=1调用函数a2，而 $(i+1)*5$ 是实参。

例11.7 有两个元素的函数指针数组的复杂说明的使用举例。

```
#include "stdio.h"
int a1(int);
int a2(int);
int main(void)          /*fpa是有2个元素的函数指针数组，每个元素指向*/
{  int i,(*fpa[2])(int); /*的函数有一个整型形参，返回整型值*/
    fpa[0]=a1;   /*第1个（下标为0）元素指向a1函数*/
    fpa[1]=a2;   /*第2个（下标为1）元素指向a2函数*/
    for(i=0;i<2;i++)
        printf("%d\n",fpa[i]((i+1)*5));/*依次调用fpa[0]、fpa[1]所指函数*/
    return 0;
}
int a1(int x){ printf("in function a1,x=%d\n",x); return 2*x; }
int a2(int y){ printf("in function a2,y=%d\n",y); return 2*y; }
```

例11.8 复杂说明

```
int (*(*(*f[2])())[2])(int, int);
```

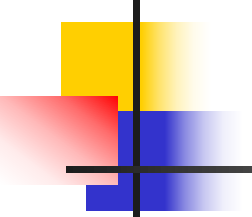
的使用举例。

分析：要能够正确使用上面声明中所说明的f，必须从相应的类型理解、赋值方法、调用方式三个方面进行把握。

1) 类型理解

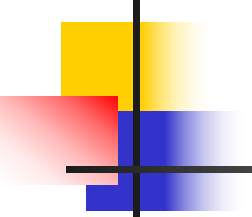
类型理解又分为f是什么？f数组元素所指的对象是什么？f数组元素所指对象的返回值所指的对象又是什么？三个层面来加以理解。

- (1) 理解声明语句中f的含义。即：f是一个有两个元素的函数指针数组；f函数指针数组中的每个函数指针元素所指向函数是无参，且返回值是指向有两个元素的指针数组的指针，该指针数组的每个指针元素指向一个有两个整型形参，且返回值为整型值的函数。
- (2) 正确构造f数组元素所指的对象。该对象是无参函数，且返回值为指向有两个元素的指针数组的指针，该指针数组的每个指针元素指向一个有两个整型形参，且返回值为整型值的函数。即由声明语句`int (*(*f1()))[2])(int,int);`和`int (*(*f2()))[2])(int,int);`所说明的函数f1和f2。
- (3) 构造函数f1和f2的返回值所指对象。该对象应该能够被指向有两个元素的函数指针数组的指针所指向。同时，该函数指针数组的每个指针元素应该指向一个有两个整型形参，且返回值为整型值的函数。声明语句`int (*pf[2][2])(int,int);`所说明的二维函数指针数组能够满足要求。即pf可以作为函数f1和f2的返回值。



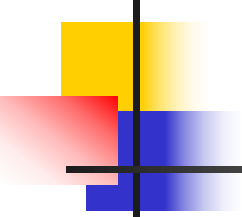
2) 赋值方法

- (1) **f**数组元素的赋值。由于**f**是一个有两个元素的函数指针数组，因此只要把满足**f**数组元素所指函数要求的函数名一一赋给**f**数组的元素即可。即：通过**f[0]=f1;**和**f[1]=f2;**就可以使**f[0]**指向函数**f1**，而**f[1]**指向函数**f2**。
- (2) **pf**数组元素的赋值。由于**pf**是二维函数指针数组，且每个数组元素可以指向一个有两个整型形参，且返回值为整型值的函数。因此，通过**pf[0][0]=add;**和**pf[0][1]=sub;**就可以使**pf[0][0]**指向加法函数**add**，**pf[0][1]**指向减法函数**sub**。同理，通过**pf[1][0]=mul;**和**pf[1][1]=div;**就可以使**pf[1][0]**指向乘法函数**mul**，**pf[1][1]**指向除法函数**div**。
- (3) **f1**函数通过**return pf;**使其返回值指向**pf**数组的第0行；**f2**函数通过**return pf+1;**使其返回值指向**pf**数组的第1行。



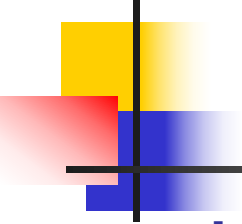
3) 调用方式

- 通过`(((*f[0]( ))[0]))(20,10);`调用add函数,其中, 20和10是实参。
- 具体分析如下: `f[0]( )`调用函数f1, 其返回值是指向pf数组第0行的指针。
- `(*f[0]( ))`即pf数组第0行(第0行的起始地址), 而`(((*f[0]( ))[0]))`表示pf数组第0行、第0列的元素, 它是指向add函数的指针。
- 因此, 再通过`(((*f[0]( ))[0]))(20,10)`调用, 就等价于`add(20,10)`。
- 对`(((*f[0]( ))[1]))(20,10);`及后面的两个调用语句可以进行类似分析。



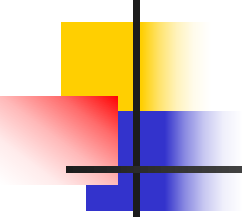
例11.8-continue

```
#include "stdio.h"
int (*( *(*f[2])())[2])(int,int);
int (*( *(f1()))[2])(int,int);
int (*( *(f2()))[2])(int,int); int (*pf[2][2])(int,int);
int add(int,int); int sub(int,int); int mul(int,int); int div(int,int);
int main(void)
{ f[0]=f1; /* 对函数指针f[0]赋值 */
  f[1]=f2; /* 对函数指针f[1]赋值 */
  ((*(*f[0]())[0]))(20,10); /* 等价于add (20,10) */
  ((*(*f[0]())[1]))(20,10); /* 等价于sub (20,10) */
  ((*(*f[1]())[0]))(20,10); /* 等价于mul (20,10) */
  ((*(*f[1]())[1]))(20,10); /* 等价于div (20,10) */
  return 0;
} 演示: 源程序\ex11_8.c
```



例11.8-continue

```
int (*(f1()))[2](int,int)
{   pf[0][0]=add; /* 对函数指针pf[0][0]赋值 */
    pf[0][1]=sub; /* 对函数指针pf[0][1]赋值 */
    printf("function f1 is called!\n");
    return pf;
}
int (*(f2()))[2](int,int)
{   pf[1][0]=mul; /* 对函数指针pf[1][0]赋值 */
    pf[1][1]=div; /* 对函数指针pf[1][1]赋值 */
    printf("function f2 is called!\n");
    return pf+1;
}
int add(int x,int y)
{   printf("%d+%d=%d\n",x,y,x+y);
    return (x+y);
}
```



例11.8-continue

```
int sub(int x,int y)
{ printf("%d-%d=%d\n",x,y,x-y);
  return (x-y);
}
int mul(int x,int y)
{ printf("%d*%d=%d\n",x,y,x*y);
  return (x*y);
}
int div(int x,int y)
{ if(y!=0){
    printf("%d/%d=%d\n",x,y,x/y);
    return (x/y); }
  else{
    printf("divide by zero! return zero!\n");
    return (0); }
}
```